MINI ERP

Project implementation summary | Laravel + Vue 3 + Node.js + MariaDB | Phases 1-5

Project purpose: Local full-stack ERP learning project demonstrating product management, inventory transactions, purchase/sales workflows, dashboards, service-to-service notifications, and role-based access.

1. Technology stack and local setup

Layer	Technology / configuration
Backend	PHP 8.5.12, Laravel REST API, Composer 2.10.3
Frontend	Vue 3, Vite, Axios; local UI at http://localhost:5173
Database	MariaDB 10.11 / MySQL-compatible; database: mini_erp; XAMPP
Notifications	Node.js 24.22.1, Express, dotenv; http://127.0.0.1:3001
Laravel API	http://127.0.0.1:8000; frontend /api requests proxied via Vite
Working directories	Backend: E:\xampp\htdocs\mini-erp Frontend: E:\xampp\htdocs\mini-erp-frontend Node: E:\xampp\htdocs\mini-erp-notifications

2. Completed modules

Phase	Implemented and checked
1 - Product API	Laravel product CRUD; validation; SKU uniqueness; name/SKU search; paginated API.
2 - Vue product UI	Product list, add/edit/delete, search debounce, pagination, error handling, confirmation.
3 - Notifications	Express low-stock service, service-token authentication, Laravel HTTP integration, Vue alerts panel.
4 - Inventory ERP	Suppliers; purchase orders/items; sales orders/items; stock movements; atomic receive/confirm flows; overselling protection; reporting API; Vue order forms and dashboard.
5 - Authentication	Login/logout UI for admin and staff; Laravel protected routes and role-based product permissions; authentication feature tests partially verified.

3. Main database entities

Products (SKU, price, cost price, stock quantity, reorder level, status); suppliers; purchase_orders and purchase_items; sales_orders and sales_items; stock_movements; users with admin/staff roles. Stock movements preserve before/after quantities and order references.

4. Transaction and business rules

- Draft purchase and sales orders do not change inventory.
- Receiving a purchase increases stock and writes an IN movement; repeated receiving returns HTTP 422.

• Confirming a sale reduces stock and writes an OUT movement; repeated confirmation and insufficient stock return HTTP 422.

• Laravel database transactions and row locks (lockForUpdate) guard stock updates and reduce overselling risk.

• Notification requests are sent after the inventory transaction commits; direct stock editing was rejected in an authentication test.

5. API and UI overview

Area	Routes / interface
Products	GET /api/products (search, pagination); GET /api/products/{id}; create/update/delete APIs.
Inventory	GET /api/inventory/summary; GET /api/inventory/movements (including IN/OUT filters).
Orders	Purchase draft creation and receive; sales draft creation and confirm; order listings/details.
Notifications	GET /api/notifications via Laravel; Node GET /health and authenticated low-stock endpoints.
Authentication	POST /login; POST /logout; GET /api/auth/me (as shown in browser network panel).
Vue navigation	Dashboard, Products, Purchases, Sales; admin/staff identity and logout visible.

6. Last observed project data

From the browser network response provided during the review:

Dashboard metric	Value
Total / active products	1 / 1
Stock units / low-stock products	8 / 0
Draft / received purchase orders	1 / 1
Draft / confirmed sales orders	0 / 2

The dashboard originally displayed zeros for some order counts because Vue used mismatched field names. Correct API keys: draft_purchase_orders, received_purchase_orders, draft_sales_orders, confirmed_sales_orders. A fix was provided; its final visual verification was not shown. Stock history showed one IN (+10) and two OUT (-1, -4) movements, leaving 8 units.

7. Test and verification record

• Phase 4 Task 1: 5/5 migration checks passed.

• Phase 4 Task 2: 6/6 Eloquent/purchase receiving checks passed.

• Phase 4 Task 3: 7/7 sales/overselling checks passed.

• Phase 4 Task 4A: 6/6 inventory reporting checks passed.

• Sales confirmation verification: 5/5 checks passed.

• AuthenticationTest initially reported 8 passed, 3 failed due to HTTP 404 on login/logout endpoints. Later screenshots show admin and staff signed into the UI; a final passing test run has not been provided.

8. Local demonstration accounts - private reference

Role	Email (verify in local DB)	Requested local password
Administrator	admin@example.test	Test@123
Staff	staff@example.test	Test@123

Important: These email addresses were example addresses used in setup instructions, not independently verified against the database. The user requested the password Test@123 for both local accounts, but final database verification of the password was not provided. Check the actual user records before relying on these credentials.

Security: This PDF contains demo login information. Keep it private and do not add it to a public GitHub repository. Replace the shared password before any deployment. Never commit Laravel .env, Node .env, service tokens, database credentials, or production secrets.

9. GitHub publishing plan

Recommended: publish the backend, frontend, and notifications service as three separate GitHub repositories, or intentionally create a new monorepo with the three projects as subfolders. The following commands assume three separate repositories and should be run independently in each directory.

Laravel backend

```
cd E:\xampp\htdocs\mini-erp
git status
git add .
git commit -m "Initial Mini ERP implementation"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/mini-erp-backend.git
git push -u origin main
```

Vue frontend

```
cd E:\xampp\htdocs\mini-erp-frontend
git status
git add .
git commit -m "Initial Mini ERP implementation"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/mini-erp-frontend.git
git push -u origin main
```

Node notification service

```
cd E:\xampp\htdocs\mini-erp-notifications
git status
git add .
git commit -m "Initial Mini ERP implementation"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/mini-erp-notifications.git
git push -u origin main
```

Use git init first only if the directory is not already a Git repository. Create the corresponding empty repositories on GitHub first. If a remote named origin already exists, inspect it using git remote -v rather than adding it again.

10. Pre-push security checklist

• Verify .gitignore excludes .env, .env.*, vendor/, node_modules/, and other generated/local files; keep safe example env templates such as .env.example.

• Run git status and git diff --cached --name-only before committing; inspect the staged content for credentials and secrets.

• Check Laravel app key, DB username/password, Node SERVICE_TOKEN, and notification service token are not tracked.

• If a secret was committed previously, adding .gitignore is not enough: rotate the secret and remove it from Git history before publishing.

• Include a README with setup commands, environment variable names (not values), architecture, endpoints, screenshots, and testing instructions.

• Run php artisan test and npm run build; confirm the three services start and the authenticated dashboard works.

11. Next enhancements

Complete final authentication test rerun and permissions audit; confirm corrected dashboard order counts; improve alert lifecycle (deduplicate and clear resolved alerts); add automated purchase/sales integration tests; implement stronger secrets management and deployment configuration.

Document status: based on the implementation steps, code excerpts, test summaries, and screenshots shared in this conversation. Not an independent repository audit.